

Reviewer Pack — Minimal Rank of Tropical Derivatives vs Resolution Proof Len...

Entry 026a0bf5b293 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 09:43:13 UTC

Minimal Rank of Tropical Derivatives vs Resolution Proof Length for XOR Tautologies

Entry ID: 026a0bf5b293 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 09:43:13 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory

Statement:

[‘For every XOR tautology φ with n variables, let $D(\varphi)$ be its tropical derivative and $r(\varphi)$ be the length of its shortest resolution proof. Then, $D(\varphi) = \Theta(r(\varphi))$ up to an additive error of $O(\log n)$.’, ‘Equivalently, for a given $\varepsilon > 0$, there exists a polynomial $p(n)$ such that for all XOR tautologies φ with n variables and all $\varepsilon > 0$, $|D(\varphi) - r(\varphi)| \leq \varepsilon * \log n$.’, ‘Additionally, if a XOR tautology φ has no resolution proof of length less than $n^{\{1/2\}}$, then $D(\varphi) = \Theta(n^{\{1/2\}})$.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework for studying algebraic objects over the tropical semiring, which may reveal subtle properties of computational problems. By examining the relationship between the tropical derivative and resolution proof length, we may uncover new invariants that separate complexity classes.’, ‘The use of tropical derivatives has been successful in bounding communication complexities and circuit depths, suggesting that they might also be useful for studying resolution proofs, a central concept in complexity theory.’]

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1c6c1e5cc2d08962

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For XOR tautologies with n variables, if the Spearman rank correlation coefficient between tropical derivative $D(\varphi)$ and proof length $r(\varphi)$ exceeds 0.7 for all seeds, AND polynomial regression yields a function $g(n, \varepsilon)$ of form $\Theta(\log n)$ with 95% confidence interval within $O(\log n)$, then support is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND complexity theory AND minimal rank tropical derivatives resolution proof length XOR tautologies - resolution proof length XOR tautology tropical geometry implications complexity theory - minimal rank tropical derivative length resolution proof XOR tautology analysis

Top relevant hits considered: - [<http://arxiv.org/abs/1610.00298v4>] Khovanskii bases, higher rank valuations and tropical geometry - [<http://arxiv.org/abs/1306.3497v2>] The number of vertices of a tropical curve is bounded by its area - [<http://arxiv.org/abs/2604.11166v2>] Asymptotic Behavior of Tropical Rank Functions - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1207.1925v1>] Introduction to tropical algebraic geometry - [<http://arxiv.org/abs/1206.1925v1>] Counting Algebraic Curves with Tropical Geometry - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_tautology(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropical_derivative(tau):
        n = len(tau)
        D = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n):
            for j in range(i + 1, n + 1):
                if tau[i] == 1 and tau[j - 1] == 1:
                    D[i][j] = 1
                elif tau[i] == 0 and tau[j - 1] == 0:
                    D[i][j] = 1
        return D
```

```

def resolution_length(tau):
    n = len(tau)
    clauses = [tau[:i] + [1] + tau[i+1:] for i in range(n)]
    length = 0
    while True:
        new_clauses = []
        added_clause = False
        for c1 in clauses:
            for c2 in clauses:
                if len(set(c1) & set(c2)) == n - 1:
                    new_clause = [x for x in c1 + c2 if x not in (0, 1)]
                    if new_clause not in new_clauses and new_clause not in clauses:
                        new_clauses.append(new_clause)
                        added_clause = True
            if not added_clause:
                break
        clauses.extend(new_clauses)
        length += len(new_clauses)
    return length

def spearman_rank_correlation(D, r):
    n = len(D)
    rank_D = [0] * n
    rank_r = [0] * n
    for i in range(n):
        rank_D[i] = sorted(range(n), key=lambda j: D[j][i])[-1]
        rank_r[i] = sorted(range(n), key=lambda j: r[j])[-1]
    return sum((rank_D[i] - rank_r[i])**2 for i in range(n)) / (n * (n**2 - 1))

def polynomial_regression(x, y):
    n = len(x)
    Sx, Sy, Sxx, Sxy = 0, 0, 0, 0
    for i in range(n):
        Sx += x[i]
        Sy += y[i]
        Sxx += x[i]**2
        Sxy += x[i] * y[i]
    a = (n * Sxy - Sx * Sy) / (n * Sxx - Sx**2)
    b = (Sy - a * Sx) / n
    return a, b

def log_bound(n):
    return math.log(n)

results = []
for _ in range(30):
    tau = generate_xor_tautology(random.randint(4, 40))
    D = tropical_derivative(tau)
    r = resolution_length(tau)
    results.append((D, r))

ranks = [spearman_rank_correlation(D, r) for D, r in results]
a, b = polynomial_regression(range(len(ranks)), ranks)
mean_rank = sum(ranks) / len(ranks)
std_rank = math.sqrt(sum((r - mean_rank)**2 for r in ranks) / len(ranks))

```

```

if all(rank > 0.7 for rank in ranks):
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "Spearman rank correlation < 0.7"

return {
    "metric_name": "Spearman Rank Correlation",
    "metric_value": mean_rank,
    "instances_tested": len(ranks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Spearman rank correlation < 0.7\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test with increased time limits to ensure it completes and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14193
2	preregistration	ollama_remote	glm4:latest	0	0	6154
3	novelty	ollama_remote	glm4:latest	0	0	4700
4	novelty	ollama_remote	glm4:latest	0	0	6001
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44104
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15270
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14467
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34397
9	judge	ollama_remote	glm4:latest	0	0	45438

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184725 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/026a0bf5b293.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/026a0bf5b293.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/026a0bf5b293.tar.gz` (if generated)